# Amazon RDS for MySQL to BigQuery Migration

## Proposed Solution

I would use **Google Cloud Datastream** to replicate the 20-million-row table from Amazon RDS for MySQL directly into BigQuery. Datastream is a managed, serverless change data capture (CDC) service that reads the MySQL binary log. It performs an initial backfill of all existing rows and then applies ongoing inserts, updates and deletes in near real time.

This approach requires no extraction scripts, no intermediate files and no staging database. Google lists Datastream as the recommended CDC method for replicating databases into BigQuery, and it can be defined entirely in Terraform.

## Assumptions

- The source runs MySQL 5.7 or 8.0 with the InnoDB engine.
- The table has a primary key (id), which Datastream uses to apply updates and deletes correctly.
- The table is approximately 5 to 20 GB, so network transfer cost is minor.
- The database owner approves the binary log configuration changes on RDS.
- Ongoing synchronization is beneficial. If only a one-time copy is required, the stream is stopped after validation.

## Why Not Other Options

- **Batch export to Cloud Storage, then a BigQuery load job:** This is a valid option for a strict one-time load and gives full control over file format and types. It requires custom extraction code and does not keep the data current. I would choose it if binary log changes on RDS were not permitted.
- **Database Migration Service to Cloud SQL, then a federated query:** This adds an extra hop and requires paying for a Cloud SQL instance only to stage the data. It is appropriate only when the application database itself is moving to Cloud SQL.
- **Manual CSV export with the mysql client and sed:** Converting tab-separated output to commas corrupts any value that contains a comma, tab or line break. CSV also loses data types, and relying on schema auto-detection over 20 million rows is a risk.
- **Third-party SaaS tools (for example, Hevo or Fivetran):** These work well but add licensing cost and route production data through an external vendor. A native Google Cloud service meets the requirement without that dependency.
- **AWS Data Pipeline:** The service has been closed to new customers since July 2024 and is in maintenance mode.

## Implementation Steps

1. **Prepare Amazon RDS.**
    - Enable automated backups, which is required for binary logging on RDS.

- In the DB parameter group, set binlog_format = ROW and binlog_row_image = FULL.
- Extend binary log retention so that logs are not purged during the backfill:

  CALL mysql.rds_set_configuration('binlog retention hours', 168);

2. **Create a dedicated replication user** with only the SELECT, REPLICATION SLAVE and REPLICATION CLIENT privileges. The password is stored in AWS Secrets Manager and Google Secret Manager and is never committed to the repository.
3. **Configure network connectivity.** Allow the Datastream regional IP addresses in the RDS security group with TLS enforced. For production, private connectivity over a site-to-site VPN is preferable.
4. **Create the Datastream resources** with Terraform (google_datastream_connection_profile for the source and destination, and google_datastream_stream). The stream includes only the mydb.orders table and has backfill enabled.
5. **Start the stream** during a low-traffic period, monitor the backfill progress, and confirm that CDC continues after the backfill completes.
6. **Validate the data** as described below before handing the table to downstream users.

### Schema and Type Mapping

Datastream creates the destination table and maps MySQL types to BigQuery types automatically. I review the resulting schema rather than accepting it without inspection, and I expose a curated view for consumers where a different type is more suitable.

| MySQL Type | BigQuery Type (Datastream) | Handling |
|---|---|---|
| INT, BIGINT | INT64 | No change required. |
| BIGINT UNSIGNED | NUMERIC | Values can exceed the INT64 maximum. Cast to INT64 in the view only if MAX(id) permits. |
| DECIMAL(p,s) | NUMERIC or BIGNUMERIC | Precision is preserved. No change required. |
| TINYINT(1) / BOOL | INT64 | Cast to BOOL in the view. |
| CHAR, VARCHAR, TEXT, ENUM | STRING | No change required. |
| SET | STRING | Split into ARRAY<STRING> in the view if required. |
| DATETIME | DATETIME | No change required. |

| MySQL Type | BigQuery Type (Datastream) | Handling |
|---|---|---|
| TIMESTAMP | TIMESTAMP | Stored in UTC. No change required. |
| TIME | INTERVAL | Documented for consumers. |
| JSON | JSON | No change required. |
| BLOB, BINARY, VARBINARY | STRING (hex encoded) | Convert with FROM_HEX() if BYTES is required. |

## Additional Considerations

- **Metadata column:** Datastream adds a datastream_metadata column to each table. Consumers should query the curated view rather than the raw table.
- **Collation:** MySQL collations are typically case-insensitive, whereas BigQuery string comparison is case-sensitive. This can change the results of DISTINCT, GROUP BY and joins, so it is documented for analysts.
- **Schema changes:** New columns on the source propagate automatically. Changes to existing column types require manual review.

### Row Count Validation

I validate the migration with Google's open-source **Data Validation Tool (DVT)**, which connects to both MySQL and BigQuery and compares them directly. Validation is performed at three levels.

1. **Row count:** The total number of rows must match between source and destination.
2. **Column aggregates:** SUM(amount) and COUNT(DISTINCT id) confirm that there are no duplicate rows and no loss of numeric precision.
3. **Row-level hash:** A hash of every column for a sample range of primary keys detects encoding or type conversion errors that counts and sums cannot detect.

```
# Row count and column aggregates

data-validation validate column \

  -sc mysql_src -tc bq_tgt \

  -tbls mydb.orders=my_project.mydb.orders \

  --count '*' --sum amount

# Row-level hash comparison on a sample range
```

```
data-validation validate row \

  -sc mysql_src -tc bq_tgt \

  -tbls mydb.orders=my_project.mydb.orders \

  -pk id --hash '*' --filters 'id < 100000'
```

Because CDC continues while validation runs, the source may change between the two counts. To obtain an exact comparison, I either run validation during a brief pause in writes or restrict both sides to rows that no longer change (for example, created_at < '<cutoff>'). I also set a low max_staleness on the BigQuery table so that the comparison reflects the latest applied changes.